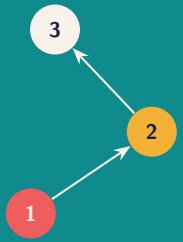
Process Mining

From Event Data to Process Insight

Sylvio Barbon Junior | Institution / Program

Course repository



What this course is really about

Observe	Explain	Improve
Turn operational records into trustworthy event data.	Reveal paths, choices, delays, rework, and deviations.	Convert evidence into a testable process intervention.

Key idea Process mining connects data to process behavior. The diagram is not the result; the defensible explanation is.

Course repository: `github.com/sbarbonjr/process_mining_course`

The learning journey

Foundation

- Process questions and event semantics
- Data quality and exploratory analysis
- Behavioral models and discovery

Application

- Model quality and conformance
- Performance and organizational views
- Object-centric analysis and delivery

Course operating model

Each unit combines

- Concepts and visual intuition
- A worked event-log example
- A PM4Py laboratory
- A short interpretation challenge

Core tools

Python, pandas, PM4Py, Jupyter, Graphviz, Git

Assessment	
Labs	30%
Concept checks	20%
Capstone analysis	35%
Presentation	15%

Discuss What process do you encounter every week that leaves a digital trace?

01

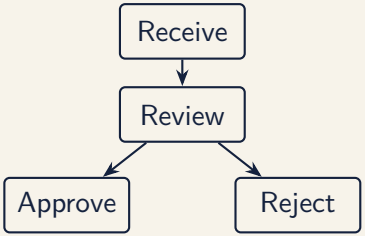
Process Thinking and Process Mining

Start with the operational question, not the algorithm.

A process is behavior over time

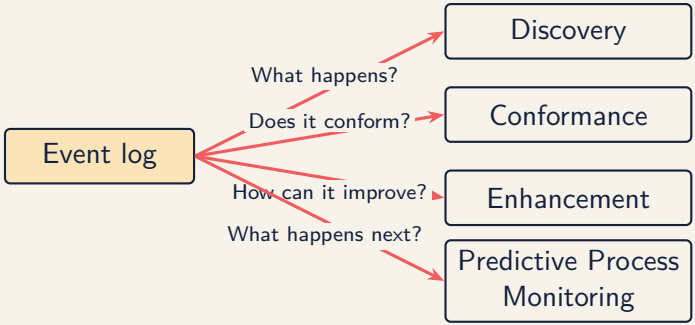
A process consists of:

- **Cases**: process instances
- **Activities**: meaningful steps
- **Events**: recorded activity executions
- **Resources**: people or systems involved
- **Outcomes**: time, cost, quality, compliance



Key idea A process model is a claim about possible behavior. An event log is evidence of observed behavior.

Four questions, four branches



Discover
Reveal actual behavior.

Check
Compare log and model.

Enhance
Add time, cost, or decisions.

Predict
Anticipate outcomes, delays, and risks.

Process mining is not a dashboard

	Typical analytics	Process mining
Unit	Rows and aggregates	Cases and ordered events
Question	How many?	How did work unfold?
Structure	Usually tabular	Sequential, concurrent, cyclic
Output	KPI or prediction	Behavior, deviation, performance
Risk	Missed context	Misleading case/event semantics

Discuss A delivery KPI worsened. What sequence-level questions would you ask before recommending a change?

Frame a useful analysis

1. **Decision**: What decision should this analysis support?
2. **Scope**: Where does the process begin and end?
3. **Case**: What is one process instance?
4. **Evidence**: Which systems record relevant events?
5. **Success**: Which behavior or outcome should change?

Example

"Why are some orders late?" becomes: "Which paths, handovers, and waiting periods distinguish orders delivered after the five-day target?"

02

Event Data and Event-Log Semantics

The quality of the analysis begins with the meaning of an event.

The minimum viable event log

Case	Activity	Timestamp	Resource	Amount
O-17	Receive order	09:02	Portal	480
O-17	Check credit	09:16	Elena	480
O-18	Receive order	09:12	Portal	125
O-17	Approve order	10:03	Marco	480
O-18	Check credit	10:24	Elena	125

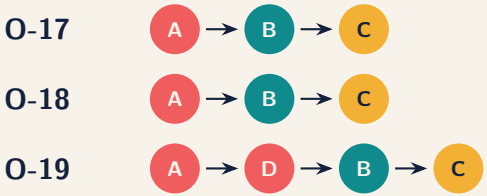
Case ID
Groups related events.

Activity
Names what happened.

Timestamp
Orders events in time.

Key idea Resources, costs, lifecycle states, and business attributes enrich analysis, but they do not repair an incoherent case notion.

From events to traces and variants



Trace: the ordered activity sequence for one case.

Variant: a distinct trace pattern shared by cases.

Here: 3 cases, 2 variants.

Lifecycle semantics change the answer

One activity may emit several events

- Start
- Suspend / resume
- Complete

Key idea Without lifecycle information, elapsed time may be mistaken for active work time.

A first PM4Py event log

```python
import pandas as pd
import pm4py

events = pd.read_csv("orders.csv")
events["timestamp"] = pd.to_datetime(
    events["timestamp"], utc=True
)

events = pm4py.format_dataframe(
    events,
    case_id="case_id",
    activity_key="activity",
    timestamp_key="timestamp",
)

pm4py.write_xes(events, "orders.xes")
```

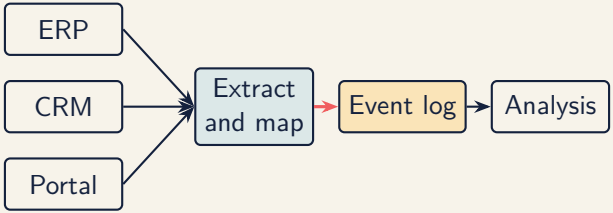
Before running discovery: verify identifiers, timestamps, ordering, duplicates, and event meaning.

03

Event-Log Engineering and Data Quality

An event log is a designed analytical artifact, not a raw database export.

From source systems to an event log



Mapping decisions include:

- Which database change counts as a business event?
- How are records correlated into cases?
- Which timestamps and time zones are authoritative?
- Which low-level events should be abstracted?

Six common data-quality traps

1. Missing events
2. Duplicate events
3. Incorrect ordering
4. Mixed granularity
5. Unstable case identifiers
6. Selection and extraction bias

Diagnostic habit

For every surprising pattern, ask: *Is this process behavior, logging behavior, or extraction behavior?*

The case-notion test

A useful case notion should satisfy three questions:

1. Do the grouped events belong to the same process instance?
2. Does the resulting sequence have a meaningful beginning and end?
3. Does it support the decision the analysis is meant to inform?

Order as case

Good for customer lead time and order completion.

Item as case

Better for partial fulfillment and item-level handling.

A compact quality report

Check	Evidence	Decision / limitation
Case completeness	8.4% lack an end event	Exclude open cases from cycle-time KPI
Timestamp order	23 negative intervals	Correct known clock offset; flag remainder
Duplicates	1.2% exact duplicates	Remove with documented key
Activity meaning	“Update” has 4 sources	Split by business semantics
Coverage	Data begins Jan 2025	No claims about earlier process versions

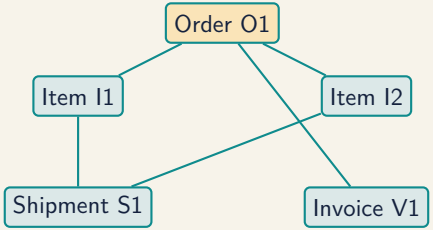
Key idea Documenting exclusions is part of the result, not administrative overhead.

Profile before you transform

Level	Profile	Decision it informs
Event	Missingness, duplicates, timestamps	Repair, exclude, or reinterpret
Case	Length, duration, completeness	Boundaries and open-case policy
Activity	Frequency, lifecycle, resources	Abstraction and filtering
Variant	Coverage, rarity, entropy	Noise strategy and segmentation
Temporal	Volume, drift, seasonality	Time windows and evaluation split

Key idea Profiling is not merely descriptive. It determines which transformations, models, and evaluation designs are defensible.

Flattening is a modeling choice



Flatten by order

- Shared item events converge

Flatten by item

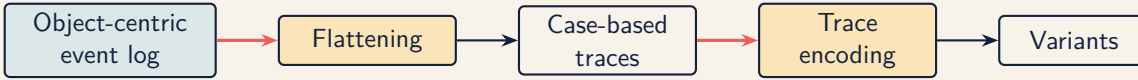
- Shared order events duplicate

Keep objects

- Relationships remain explicit

Discuss Which case notion preserves the behavior needed by your question, and which frequencies does it distort?

Flattening and encoding: two layers, two decisions



Flattening

Decides **which events belong to the same case**. Coarser, earlier: it chooses the case notion (order, item, shipment...) and creates convergence or divergence before any trace exists.

Encoding

Decides **how an already-defined case is represented** as a trace: control-flow, lifecycle, time, or payload. Finer, later: it shapes variants once cases already exist.

Key idea Both are modeling choices, not neutral defaults: flattening fixes what counts as one case, encoding fixes how that case's history is read.

04

Exploratory Process Analysis

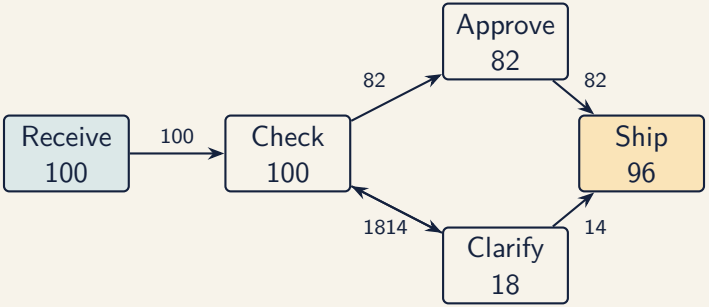
Build a map of the evidence before building a process model.

Explore at four levels

Log	Case	Activity	Variant
Cases, events, date range, coverage	Duration, event count, outcomes	Frequency, resources, timing	Paths, prevalence, performance

Discuss Which level would reveal rework? Which would reveal a seasonal extraction gap?

A directly-follows graph



A DFG summarizes observed adjacency:

- Nodes are activities; edges mean "directly followed by"
- Counts can represent frequency or performance
- Loops, choices, and concurrency require careful interpretation

Variants: useful, but easy to misuse

Rank	Variant	Cases	Median time
1	Receive, Check, Approve, Ship	68%	2.1 d
2	Receive, Check, Clarify, Check, Ship	14%	5.8 d
3	Receive, Check, Reject	8%	1.4 d
4+	17 other variants	10%	8.2 d

Variants reveal

- Dominant routes
- Rework patterns
- Exceptional behavior

Variants can hide

- Concurrent behavior
- Attribute differences
- Important rare cases

A variant depends on its trace encoding

Encoding	Trace representation	Variants distinguish
Control-flow	Activity sequence	Routing differences
Lifecycle-aware	Activity + transition	Start, suspend, resume, complete
Time-aware	Activity + duration bucket	Behavioral and timing regimes
Payload-aware	Activity + selected attributes	Context or outcome cohorts
Learned	Sequence / payload embedding	Similarity beyond exact equality

Key idea Variant counts are not properties of the raw log alone. They are produced by an encoding, equivalence rule, and similarity threshold.

Profile the payload before using it

Event payload

Information carried by an event beyond case, activity, and timestamp.

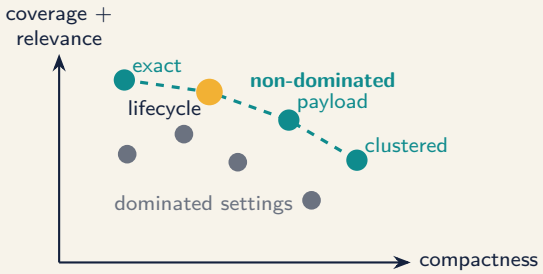
- Structured attributes and measurements
- Text, documents, images, audio, and sensor references
- Resource, cost, location, and product context

Profile before encoding

- Availability at event time
- Missingness and cardinality
- Drift, semantics, and cardinality
- Privacy, proxies, and future leakage

Discuss Use payload dimensions only when they serve the analytical question; otherwise they can fragment traces into meaningless one-case variants.

Pareto front for trace-variant detection



Candidate settings
Encoding, similarity threshold, rare-variant cutoff, and payload dimensions.

Objectives
Case coverage, compactness, decision relevance, and temporal stability.

Inspect representative traces before selecting a non-dominated setting.

A modeling-readiness profile

Before discovery or prediction, record:

- Case and trace-length distributions
- Variant coverage and long-tail behavior
- Class and outcome imbalance
- Missing and high-cardinality attributes
- Process and population drift
- Open versus completed cases
- Observation and label timestamps
- Potential future-information leakage

Deliverable

A one-page readiness report: usable population, known distortions, representation choice, split strategy, and excluded information.

PM4Py exploration

```
import pm4py

print(pm4py.get_start_activities(log))
print(pm4py.get_end_activities(log))

variants = pm4py.get_variants_as_tuples(log)
dfg, starts, ends = pm4py.discover_dfg(log)

filtered = pm4py.filter_case_performance(
    log,
    min_performance=5 * 24 * 60 * 60,
)
```

Interpretation task: compare common cases with slow cases. Which path differences are plausible explanations, and which need more evidence?

05

Process-Modeling Foundations

Different representations answer different questions.

Four behavioral building blocks

Sequence

$A \to B$

One follows another.

Choice

$A \to (B|C)$

One branch is selected.

Parallel

$A \to (B \parallel C)$

Both occur, order varies.

Loop

$A \to B \to A$

Behavior repeats.

Key idea A useful model must distinguish choice from concurrency and permit repetition without permitting everything.

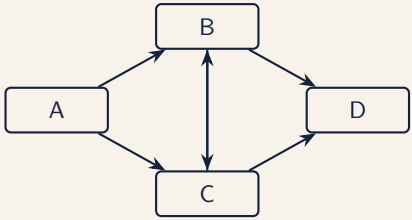
A DFG is not a complete process model

Observed traces

$\langle A, B, C, D \rangle$
$\langle A, C, B, D \rangle$

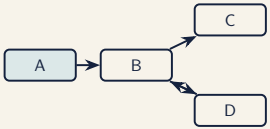
Evidence suggests B and C may be parallel.

DFG



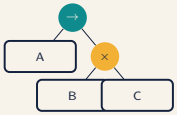
The DFG also appears to allow repeated alternation between B and C. That behavior was never observed.

Model representations



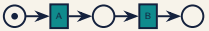
DFG

Strength: fast, readable overview
Watch for: no complete execution semantics



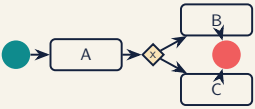
Process tree

Strength: structured, sound by construction
Watch for: restricts model shape



Petri net

Strength: precise concurrency and replay
Watch for: harder for non-specialists



BPMN

Strength: familiar business notation
Watch for: semantics vary by usage

Discuss Which representation would you use for stakeholder discussion? Which for alignment-based conformance checking?

Choose a modeling strategy

Purpose	Strategy	Representation	Emphasize
Explain flow	Structured discovery	Process tree / BPMN	Simplicity, fitness
Audit rules	Normative modeling	Petri net / Declare	Soundness, diagnostics
Explore complexity	Frequency-aware discovery	DFG / heuristics net	Coverage, readability
Predict state	State or prefix model	Transition / features	Generalization, utility
Communicate	Simplify and annotate	BPMN / DFG	Interpretability

Key idea There is no context-free "best model." State the purpose before selecting the algorithm, representation, and quality criteria.

Abstraction before algorithm

Useful strategies

- Collapse technical events
- Separate process populations
- Decompose by lifecycle or subprocess

Practice

- Version every mapping
- Compare before and after profiles
- Retain a traceable raw-event link

Soundness: can every case finish properly?

For a workflow model, we usually want:

- One meaningful start and one meaningful completion
- Every modeled activity can occur
- No deadlocks before completion
- No tokens or work left behind after completion

Why it matters

An attractive diagram can still encode impossible or incomplete behavior. Formal semantics let us test the model rather than trust its appearance.

06

Process Discovery

Discovery algorithms compress event behavior into an explanatory model.

Discovery is a balancing act

Discovery depends on:

- The event log and its abstraction level
- The algorithm's behavioral assumptions
- Noise and frequency thresholds
- The intended use of the resulting model

Three discovery families

Alpha Miner

Uses ordering relations to infer causal, parallel, and choice behavior.

Best as a conceptual foundation.

Heuristics Miner

Uses frequency and dependency measures to tolerate noise.

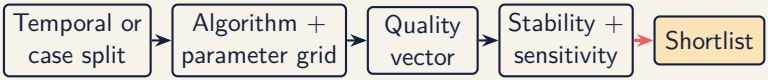
Useful for exploratory models.

Inductive Miner

Recursively finds sequence, choice, parallel, and loop cuts.

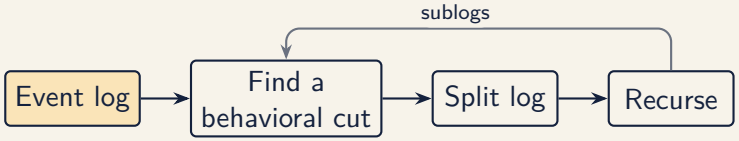
Produces sound, structured models.

A disciplined discovery experiment



1. Define constraints and metric directions before seeing results.
2. Keep discovery data separate from final evaluation data.
3. Retain all candidate parameters and artifacts, not only the winner.
4. Inspect unstable activities and edges, not only aggregate scores.

Inductive discovery intuition



Candidate cuts:

Sequence	Exclusive choice	Parallel	Loop
$\rightarrow$	$\times$	$\wedge$	$\circlearrowleft$

Key idea The result is a process tree that can be translated into a Petri net or BPMN model.

Discover with PM4Py

```python
import pm4py

# Structured discovery
tree = pm4py.discover_process_tree_inductive(
    log, noise_threshold=0.2
)
net, initial, final = pm4py.convert_to_petri_net(tree)

# Business-facing representation
bpmn = pm4py.convert_to_bpmn(tree)
pm4py.save_vis_bpmn(bpmn, "discovered_process.svg")
```

Experiment: vary the noise threshold. Record which behavior disappears, which remains, and whether the model becomes more useful.

07

Model Quality and Evaluation

No single score can tell you whether a model is fit for purpose.

The four quality dimensions

Fitness	Precision	Generalization	Simplicity
Can the model replay observed behavior?	Does it avoid allowing unseen behavior?	Does it handle plausible future behavior?	Can people understand and use it?

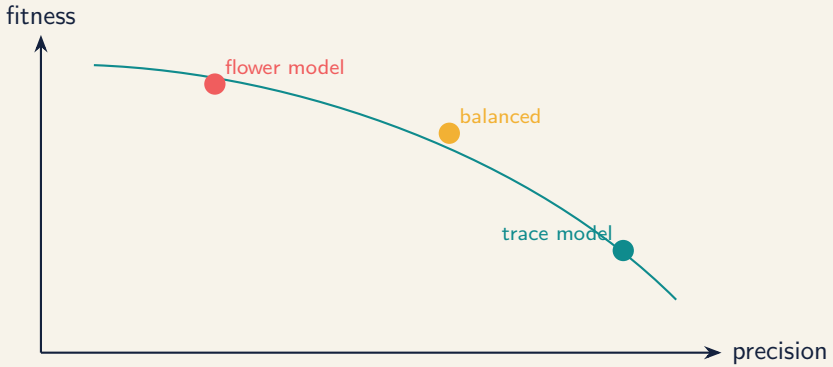
Key idea Perfect fitness is trivial if the model allows every possible trace. Quality emerges from the trade-off.

Build a quality vector, not one score

Dimension	Example measure	Interpretation risk
Fitness	Alignment / replay fitness	Rewards permissive models
Precision	Escaping-edge / alignment precision	Sensitive to log coverage
Generalization	Holdout replay, cross-validation	Depends on split design
Simplicity	Nodes, arcs, control-flow complexity	Small is not always understandable
Stability	Bootstrap edge / relation frequency	Aggregate scores can hide churn
Feasibility	Soundness, runtime, memory	Hard constraints, not preferences

Key idea Record metric direction, computation method, dataset split, and uncertainty for every component of the quality vector.

Fitness and precision pull in opposite directions



The “best” point depends on whether the model is for explanation, simulation, compliance, or prediction.

Why a weighted average can mislead

Suppose:

$$score = 0.4\, fitness + 0.4\, precision + 0.2\, simplicity$$

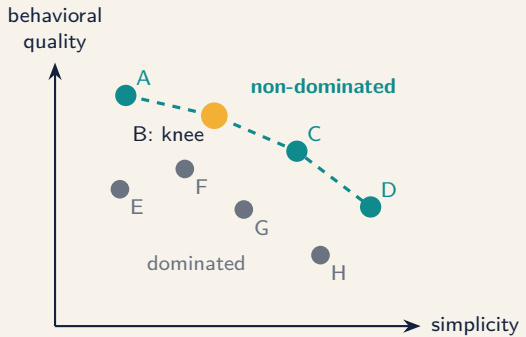
What it hides

- A critical minimum may be violated
- Different weights reverse the ranking
- Compensating metrics may be unacceptable
- Scale and normalization alter the result

Better first step

- Apply hard feasibility constraints
- Remove dominated candidates
- Inspect the Pareto front
- Add preferences only afterward

Pareto dominance and the Pareto front



Candidate X dominates Y when:

- X is no worse on every objective
- X is better on at least one objective

The **Pareto front** contains candidates not dominated by any other candidate.

Key idea A knee point offers a strong trade-off, but it is a candidate for discussion, not an automatic winner.

Practical Pareto-front selection

1. **Constrain first**: require soundness, minimum fitness, and an acceptable runtime or model size.
2. **Estimate uncertainty**: bootstrap cases or repeat temporal folds; do not trust tiny score differences.
3. **Inspect sensitivity**: vary thresholds, activity abstraction, and metric implementation.
4. **Review the front**: show all non-dominated models and explain what is gained and lost between adjacent candidates.
5. **Select transparently**: use a knee point, lexicographic rule, or stakeholder utility after the front is known.
6. **Validate once**: report the final choice on untouched holdout data.

Replay and alignments

Token-based replay

Replays traces on a Petri net and counts missing, remaining, consumed, and produced tokens.

Fast and diagnostic, but can mask some deviations.

Alignments

Find a minimum-cost correspondence between log behavior and model behavior.

More precise, but computationally more expensive.

Discuss Would you evaluate a compliance model and an exploratory model with the same tolerance for deviations? Why?

Evaluate candidates

```
fitness = pm4py.fitness_alignments(
    log, net, initial, final
)
precision = pm4py.precision_alignments(
    log, net, initial, final
)

print(fitness["log_fitness"])
print(precision)
```

A defensible comparison records:

- Scores and runtime
- Model size and readability
- Training versus held-out performance
- Which behavior matters to the analytical question

A defensible evaluation protocol

Report:

- Candidate-generation rules and excluded candidates
- Full metric vectors with uncertainty, not only the selected model
- Pareto-front membership and the final decision rule
- Holdout performance, known instability, and intended model use

08

Conformance Checking

Find the smallest meaningful explanation of how reality and the model differ.

What counts as non-conformance?

The model may be normative

- Policy
- Regulation
- Standard operating procedure

A deviation may indicate risk.

The model may be descriptive

- Earlier discovery
- Expected operating pattern
- Simulation baseline

A deviation may indicate change.

Key idea Conformance identifies differences. Domain context determines whether those differences are harmful, helpful, or simply missing from the model.

Reading an alignment

Position	1	2	3	4	5
Log	Receive	Check	Clarify	Approve	Ship
Model	Receive	Check	$\gg$	Approve	Ship
Move	sync	sync	log	sync	sync

Synchronous move
Event and model agree.

Move on log
Observed event not matched.

Move on model
Expected behavior not observed.

From deviations to findings

1. **Locate**: Which activities, paths, and cases deviate?
2. **Quantify**: How frequent, costly, or delayed are they?
3. **Segment**: Which products, teams, or periods differ?
4. **Validate**: Is the model current and the data complete?
5. **Explain**: What operational mechanism could produce this?

Strong finding

"Manual credit review is skipped in 7.3% of high-value orders, concentrated in one channel after the April system release."

Alignment diagnostics in PM4Py

```
alignments = pm4py.conformance_diagnostics_alignments(
    log, net, initial, final
)

for result in alignments[:3]:
    print(result["fitness"])
    print(result["alignment"])
```

Lab output: a ranked deviation table containing pattern, affected cases, frequency, performance impact, and a validated interpretation.

09

Performance and Process Enhancement

A bottleneck is a mechanism, not merely a red number.

Decompose elapsed time

Throughput time
Case start to completion.

Service time
Time actively worked.

Waiting time
Time between activities.

Four bottleneck signatures

- Long waiting before one activity
- Growing queue over time
- Rework returning to the same activity
- Excess handovers before completion

Do not stop at

"Activity X has the longest average duration."

Ask whether the delay comes from capacity, batching, policy, missing information, prioritization, or measurement.

Compare cohorts, not only averages

Cohort	Median cycle time	Rework rate	Cases
Standard orders	2.2 d	6%	4,180
Custom orders	6.8 d	29%	640
Custom + partner channel	9.4 d	41%	185

Improvement hypothesis

Partner-channel custom orders arrive with incomplete specifications, triggering clarification loops. A structured intake form should reduce rework and waiting time.

A hypothesis names a mechanism and predicts a measurable change.

From insight to intervention

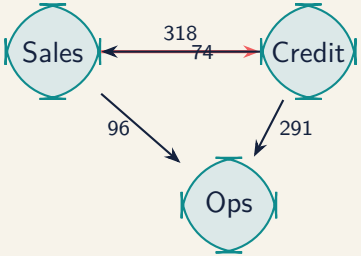
Key idea Process mining supports improvement by narrowing and testing explanations. It does not establish causality by itself.

10

Organizational and Multi-dimensional Analysis

Processes are networks of coordination, not only sequences of activities.

A handover-of-work network



Network analysis can reveal:

- Frequent handovers and coordination loops
- Central or overloaded roles
- Segregation-of-duties concerns
- Differences between formal and actual collaboration

Combine perspectives

Control flow
Which path?

Time
How long?

Organization
Who coordinated?

Data
Under which conditions?

Example synthesis
Clarification loops are slowest when cases cross from partner sales to central operations, especially for custom products with missing dimensions.

From an event log to machine-learning rows

Case-level row
One completed case, usually for retrospective outcome analysis.

Prefix-level row
One case observed up to a prediction point, used for online monitoring.

Key idea Define the observation cutoff before computing any feature or label.

Encoding strategies

Encoding	Representation	Strength	Main risk
Aggregation	Counts, durations, last value	Simple and strong baseline	Loses order
Index-based	Activity/attribute by position	Preserves early order	Wide, sparse tables
Sequence	Tokens or learned embeddings	Captures order and context	Data and compute demand
Process-aware	Variant, state, alignment moves	Uses domain structure	Model-dependent bias
Graph	DFG/object graph features	Captures relationships	Complex validation
Object-centric	Features by object type	Avoids premature flattening	Aggregation choices remain

Discuss Start with an aggregation baseline. Add a richer encoding only when it improves validation utility enough to justify complexity.

Feature provenance is part of the model

For every feature, record:

- Source event or object attribute
- Aggregation and missing-value rule
- Earliest time at which the feature is available
- Case/object population to which it applies
- Fitted preprocessing parameters and training period

Process-aware examples

Rework count, elapsed time, current activity, variant prefix, alignment cost, model state, handover count, open-object count, and workload at arrival.

Responsible resource analysis

- Analyze roles or teams before individuals where possible
- Check workload, case complexity, and data coverage
- Avoid ranking people using unvalidated process metrics
- Distinguish process constraints from individual performance
- Apply access controls, minimization, and purpose limitation

Key idea Event data reflects the system that assigns work. It is rarely a fair standalone measure of an individual's contribution.

Correlation is a clue, not a cause

A team may appear slower because it receives:

- More complex cases
- Cases already delayed upstream
- Work with poorer data quality
- Cases requiring additional controls

Discuss What additional data or study design would help distinguish a team effect from case-mix differences?

11

Object-Centric Process Mining

Some processes do not have one natural case identifier.

The flattening problem

One order may contain

- Several items
- Several deliveries
- Several invoices
- One or more payments

Forced order case

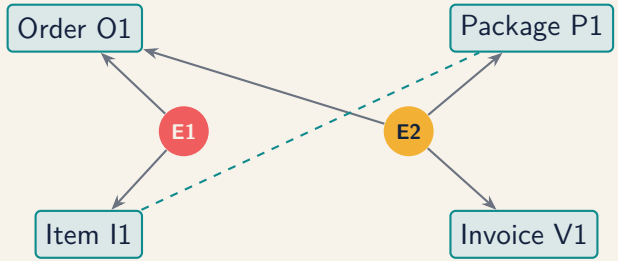
Item-level events may be duplicated across the flattened trace.

Forced item case

Shared invoice and delivery events may be duplicated instead.

This creates **convergence** and **divergence**: misleading frequencies and apparent behavioral relations.

Object-centric event data



OCEL 2.0 represents:

- Events linked to one or more typed objects
- Object-to-object relationships
- Qualified relationships and changing object attributes

Case-centric or object-centric?

Situation	Case-centric	Object-centric
One stable business instance	Strong fit	Often unnecessary
Several interacting entities	Distorting	Strong fit
Established KPI by case	Direct	Requires aggregation choices
Familiar tooling	Mature	Emerging
Relationship questions	Limited	Native

Key idea Object-centric analysis does not eliminate modeling choices. It makes relationships explicit so those choices are visible.

Object-centric features without premature flattening

Per object type

- Counts and lifecycle states
- Age and elapsed-time summaries
- Completed and open related objects
- Relationship cardinalities

At prediction time

- Freeze the object graph at the cutoff
- Aggregate only visible objects
- Preserve identifiers for grouped splits
- Test sensitivity to aggregation rules

Key idea Flatten late and explicitly. The target object and prediction time should determine which relationships become features.

Inspect an OCEL with PM4Py

```
import pm4py

ocel = pm4py.read_ocel2_json("order_management.jsonocel")

print(ocel.events.head())
print(ocel.objects.head())
print(ocel.relations.head())

ocdfg = pm4py.discover_ocdfg(ocel)
pm4py.save_vis_ocdfg(ocdfg, "object_centric.svg")
```

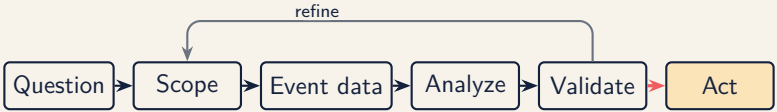
Lab comparison: flatten the same data by order and by item. Which frequencies or paths become misleading?

12

Projects and Communication

A process-mining project succeeds when evidence changes a decision.

The project lifecycle



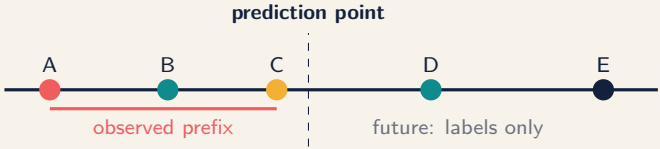
The lifecycle is iterative. Domain validation may change the event mapping, case notion, model, metric, or even the original question.

Predictive process monitoring tasks

Question	ML formulation	Example target	Useful baseline
What happens next?	Classification	Next activity	Most frequent transition
How long remains?	Regression / survival	Remaining time	Median by current state
How will it end?	Classification	Late / rejected / compliant	Base-rate model
Is this unusual?	Anomaly detection	Deviant prefix	Frequency / conformance score

Key idea The prediction must correspond to an operational decision and a time when someone can still act.

Prefixes, labels, and prediction points



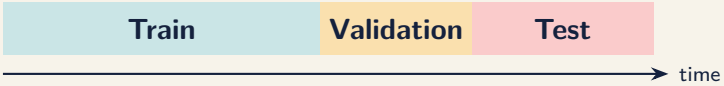
Define

- Prefix rule and cutoff
- Prediction frequency
- Label horizon

Avoid

- Features computed after cutoff
- Labels unavailable in operation
- Duplicated prefixes across splits

Split by time, group by case



- Assign every case and all its prefixes to exactly one split.
- Fit encoders, imputers, scalers, and vocabularies on training data only.
- Prefer temporal evaluation when the deployed model predicts future cases.
- Keep related customers, orders, or objects together when dependence exists.
- Reserve the test period for one final estimate after tuning.

Common leakage paths

Future leakage

- Final case attributes in a prefix
- Total event count or final duration
- Post-outcome resource or status

Split leakage

- Prefixes from one case in several splits
- Random event-level splitting

Preprocessing leakage

- Global normalization or imputation
- Variants built from all periods
- Discovery model fitted on test cases

Operational leakage

- Feature not available at scoring time
- Label definition changes after deployment

Evaluate beyond predictive accuracy

Discrimination	Calibration	Timeliness	Utility
AUC, PR-AUC, MAE, concordance	Reliability, Brier score, interval coverage	Earliness, stability across prefixes	Cost, workload, prevented delay, decision value

Also monitor:

- Performance by cohort and process state
- Feature, label, and process drift
- Alert volume, false-alarm burden, and intervention capacity

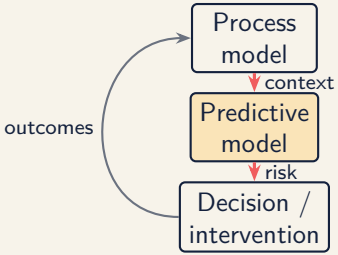
Integration patterns: process mining + ML

Process mining informs ML

Variants, model state, alignments, conformance, rework, and handovers become features or cohorts.

ML informs process analysis

Predictions identify risky prefixes and reveal where errors concentrate in the process.



Key idea Version and monitor the event mapping, process model, encoding, and predictive model as one analytical system.

An evidence chain for every finding

1. **Question**: what decision is at stake?
2. **Observation**: what pattern is measured?
3. **Interpretation**: what mechanism may explain it?
4. **Validation**: what did domain experts or other data confirm?
5. **Action**: what change should be tested?
6. **Metric**: how will we know whether it worked?

Key idea Separate observation from interpretation. It makes the analysis easier to challenge, improve, and trust.

Communicate one finding well

Lead with

- Operational consequence
- Magnitude and affected population
- Concise visual evidence
- Recommended next action

Always include

- Data scope
- Key assumptions
- Uncertainty
- Important alternatives

Headline pattern

What happens + where + impact: "Clarification loops add 4.1 days to partner-channel custom orders."

Reproducibility checklist

- Versioned extraction and transformation code
- Data dictionary and event semantics
- Fixed parameters and package versions
- Deterministic figures and tables
- Documented exclusions and filters
- Stored model artifacts
- Decision and validation log
- Privacy-aware sharing strategy

Discuss Could another analyst reproduce your central finding without asking what you clicked?

Research opportunities

Richer event data

- Multimodal event logs
- Automated event abstraction
- Object-centric, streaming, and uncertain events

Richer analysis

- Multimodal representation learning
- Causal process mining
- Prescriptive monitoring
- Privacy-preserving analytics

Multimodal event logs

Align structured events with payload such as free text, documents, images, audio, and sensor streams while preserving time, provenance, and access rules.

Key idea Each modality must add valid, timely, and governable process evidence.

Capstone: end-to-end process study

Required analysis

- Scope and analytical questions
- Event semantics and quality report
- Representation, flattening, and encoding
- Exploration, performance, and discovery
- Quality vector and Pareto-front comparison
- Conformance or deviation analysis

Required argument

- One validated finding
- Predictive baseline when relevant
- Leakage-safe split and evaluation
- One improvement hypothesis
- Ethical and analytical limitations
- Reproducible code and artifacts
- Stakeholder-oriented presentation

Capstone evaluation

Dimension	Evidence of strong work
Question and scope	Specific decision, coherent case notion, explicit boundaries
Representation	Validated semantics, justified flattening and encoding
Model quality	Full metric vectors, Pareto analysis, stable selection rule
Prediction	Leakage-safe split, calibrated errors, operational baseline
Interpretation	Findings distinguish evidence from explanation
Reproducibility	Analysis runs from documented inputs to outputs
Communication	Consequence, uncertainty, and next action are clear

The analyst's discipline

1. Question the event semantics.
2. Make representation choices explicit.
3. Compare quality vectors, not one score.
4. Prevent leakage before training.
5. Turn findings and predictions into testable actions.

Key idea The most important process-mining skill is knowing what the data can support you in saying.

Questions become evidence.

Evidence becomes better process decisions.

References — core textbook

Main reference

van der Aalst, W.M.P. (2016). *Process Mining: Data Science in Action* (2nd ed.). Springer.

This textbook, by the founder of the process mining field, is the primary reference for the concepts covered throughout the course: event log semantics, process discovery, conformance checking, and process enhancement.

References — selected articles

1. de Alvarenga, S.C., Barbon Jr., S., Miani, R.S., Cukier, M., Zarpelão, B.B. (2018). Process mining and hierarchical clustering to help intrusion alert visualization. *Computers & Security*, 73, 474–491. https://www.sciencedirect.com/science/article/pii/S0167404817302584
2. Ceravolo, P., Tavares, G.M., Barbon Jr., S., Damiani, E. (2020). Evaluation Goals for Online Process Mining: A Concept Drift Perspective. *IEEE Transactions on Services Computing*. https://ieeexplore.ieee.org/abstract/document/9124702
3. Tavares, G.M., Oyamada, R.S., Barbon Jr., S., Ceravolo, P. (2023). Trace encoding in process mining: A survey and benchmarking. *Engineering Applications of Artificial Intelligence*, 126, 107028. https://www.sciencedirect.com/science/article/pii/S0952197623012125
4. da Silva, M.C., Tavares, G.M., Gritti, M.C., Ceravolo, P. (2023). Using Process Mining to Reduce Fraud in Digital Onboarding. *FinTech*, 2(1), 120–137. https://www.mdpi.com/2674-1032/2/1/9